10 PYTHON PROGRAMS

PRACTICAL RECORD

Problem Statements • Algorithms • Flowcharts • Source Code • Sample I/O • Results

Topics covered

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

Note: Financial rules, tariffs, fees, tax rates, and concessions are illustrative classroom assumptions. Replace them with values supplied by your instructor.

1. University Scholarship Allocation

Problem statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and family income. Rank eligible applicants and resolve ties by marks, then attendance, then lower income.

Algorithm

1. Read applicant details and subject marks.

2. Calculate aggregate percentage.

3. Check every subject minimum, attendance threshold, and income limit.

4. Keep eligible applicants; sort by aggregate (descending), attendance (descending), income (ascending).

5. Display ranking and eligibility results.

Flowchart (text representation)

Start → Read rules and applicants → Calculate aggregate → Check eligibility → Sort eligible list → Display ranking → End

Python source code

```
# Sample policy: subject >= 40, attendance >= 75%, income <= 300000
SUBJECT_MIN = 40
ATTENDANCE_MIN = 75
INCOME_MAX = 300000

applicants = [
    {"name": "Asha", "marks": [88, 91, 84], "attendance": 92, "income": 180000},
    {"name": "Ravi", "marks": [78, 82, 80], "attendance": 86, "income": 250000},
    {"name": "Mina", "marks": [90, 35, 95], "attendance": 95, "income": 150000},
]

eligible = []
for a in applicants:
    aggregate = sum(a["marks"]) / len(a["marks"])
    a["aggregate"] = aggregate
    a["eligible"] = (min(a["marks"]) >= SUBJECT_MIN
                     and a["attendance"] >= ATTENDANCE_MIN
                     and a["income"] <= INCOME_MAX)
    if a["eligible"]:
        eligible.append(a)

eligible.sort(key=lambda a: (-a["aggregate"], -a["attendance"], a["income"]))
print("Eligible scholarship ranking")
for rank, a in enumerate(eligible, start=1):
    print(f"{rank}. {a['name']} | Marks: {a['aggregate']:.2f}% "
          f"| Attendance: {a['attendance']}% | Income: Rs.{a['income']}")
for a in applicants:
    if not a["eligible"]:
        print(a["name"], "- Not eligible")
```

Sample input

Sample applicants are included in the program: Asha, Ravi, Mina.

Sample output

```
Eligible scholarship ranking\n1. Asha | Marks: 87.67% | Attendance: 92% | Income: Rs.180000\n2. Ravi | Marks: 80.00% | Attenda
```

Result

The program filters applicants against all policy conditions and ranks eligible candidates using the stated tie-breaking order.

2. Progressive Electricity Billing

Problem statement

Calculate monthly household bills with progressive units slabs, fixed charge, surcharge, and rebate. Compare bills and flag a large month-to-month increase.

Algorithm

1. Read monthly consumption values.
2. Compute energy charge slab by slab.
3. Add fixed charge and surcharge; apply eligible rebate.
4. Compare each month with the previous month.
5. Print bills and flag increases greater than the chosen threshold.

Flowchart (text representation)

Start → Read monthly units → Calculate slab charge → Add charges/apply rebate → Compare months → Print bills/alerts → End

Python source code

```
def energy_charge(units):
    # Illustrative rates; replace with your supplied tariff.
    remaining = units
    charge = 0
    for slab_limit, rate in [(100, 2.0), (100, 3.0), (None, 5.0)]:
        slab_units = remaining if slab_limit is None else min(remaining, slab_limit)
        charge += slab_units * rate
        remaining -= slab_units
        if remaining <= 0:
            break
    return charge

months = {"June": 90, "July": 130, "August": 210}
fixed_charge = 50
surcharge_rate = 0.05
rebate = 20
bills = {}

for month, units in months.items():
    base = energy_charge(units)
    surcharge = base * surcharge_rate
    total = max(0, base + fixed_charge + surcharge - rebate)
    bills[month] = total
    print(f"{month}: {units} units | Bill = Rs.{total:.2f}")

names = list(months)
for i in range(1, len(names)):
    prev, current = bills[names[i-1]], bills[names[i]]
    increase = (current - prev) / prev * 100 if prev else 0
    if increase > 30:
        print(f"ALERT: {names[i]} bill increased by {increase:.1f}%")
```

Sample input

June: 90 units; July: 130 units; August: 210 units. Tariff values are illustrative.

Sample output

```
June: 90 units | Bill = Rs.239.00\nJuly: 130 units | Bill = Rs.477.00\nAugust: 210 units | Bill = Rs.897.00\nALERT: July bill
```

Result

The bill is calculated progressively, then month-to-month increases above 30% are reported. Tariff values are examples, not an official utility tariff.

3. Hospital Billing and Patient Classification

Problem statement

Total consultation, diagnostic, and treatment charges; apply a concession under a sample policy, validate inputs, and classify payment status.

Algorithm

1. Read patient category and service charges.
2. Reject negative charges or payment.
3. Calculate gross bill and eligible concession.
4. Calculate net payable and balance.
5. Classify as Paid, Partially Paid, or Unpaid.

Flowchart (text representation)

Start → Read patient and charges → Validate → Total charges → Apply concession → Determine balance/status → Print bill → End

Python source code

```
patient_type = "senior"   # general, senior, or eligible
consultation = 500
diagnostics = 1200
treatment = 3500
paid = 3000

charges = [consultation, diagnostics, treatment, paid]
if any(value < 0 for value in charges):
    raise ValueError("Charges and payment cannot be negative")

gross = consultation + diagnostics + treatment
# Illustrative concession policy: senior patients receive 10%.
discount_rate = 0.10 if patient_type == "senior" else 0
discount = gross * discount_rate
net = gross - discount
balance = max(0, net - paid)
refund = max(0, paid - net)

if paid >= net:
    status = "Paid"
elif paid > 0:
    status = "Partially Paid"
else:
    status = "Unpaid"

print("Hospital Bill")
print(f"Gross: Rs.{gross:.2f}")
print(f"Concession: Rs.{discount:.2f}")
print(f"Net payable: Rs.{net:.2f}")
print(f"Paid: Rs.{paid:.2f} | Balance: Rs.{balance:.2f}")
if refund:
    print(f"Refund due: Rs.{refund:.2f}")
print("Classification:", status)
```

Sample input

Patient type: senior; consultation Rs.500; diagnostics Rs.1200; treatment Rs.3500; paid Rs.3000.

Sample output

```
Hospital Bill\nGross: Rs.5200.00\nConcession: Rs.520.00\nNet payable: Rs.4680.00\nPaid: Rs.3000.00 | Balance: Rs.1680.00\nClas
```

Result

The program validates amounts, applies the sample senior concession, and classifies the account by payment progress. Follow the hospital's actual policy in real use.

4. Employee Payroll and Deduction Analysis

Problem statement

Calculate gross pay from basic salary, allowances, overtime, and incentives; apply progressive deductions and print a salary statement.

Algorithm

1. Read salary components and validate nonnegative values.
2. Calculate overtime pay and gross salary.
3. Calculate progressive tax using the sample brackets.
4. Subtract deductions to get net salary.
5. Print a formatted salary statement.

Flowchart (text representation)

Start → Read pay components → Calculate gross → Compute progressive deduction → Calculate net → Print statement → End

Python source code

```
basic = 30000
allowances = 8000
overtime_hours = 6
overtime_rate = 250
incentive = 2000

if min(basic, allowances, overtime_hours, overtime_rate, incentive) < 0:
    raise ValueError("Pay components cannot be negative")

overtime = overtime_hours * overtime_rate
gross = basic + allowances + overtime + incentive

# Illustrative deduction bands on gross salary.
def progressive_deduction(amount):
    tax = 0
    bands = [(20000, 0.00), (20000, 0.05), (float("inf"), 0.10)]
    remaining = amount
    for width, rate in bands:
        taxable = min(remaining, width)
        tax += taxable * rate
        remaining -= taxable
        if remaining <= 0:
            break
    return tax

deduction = progressive_deduction(gross)
net = gross - deduction
print("SALARY STATEMENT")
print(f"Basic salary : Rs.{basic:,.2f}")
print(f"Allowances   : Rs.{allowances:,.2f}")
print(f"Overtime     : Rs.{overtime:,.2f}")
print(f"Incentive    : Rs.{incentive:,.2f}")
print(f"Gross salary : Rs.{gross:,.2f}")
print(f"Deduction    : Rs.{deduction:,.2f}")
print(f"Net salary   : Rs.{net:,.2f}")
```

Sample input

Basic Rs.30000; allowances Rs.8000; 6 overtime hours at Rs.250/hour; incentive Rs.2000.

Sample output

```
SALARY STATEMENT\nBasic salary : Rs.30,000.00\nAllowances   : Rs.8,000.00\nOvertime     : Rs.1,500.00\nIncentive    : Rs.2,000
```

Result

The salary statement displays gross pay, the sample progressive deduction, and net salary. Actual payroll must use applicable statutory rules.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement

Compare mobile plans using monthly usage, validity, and budget constraints; include excess usage charges and select the least expensive eligible plan.

Algorithm

1. Read expected data usage and budget.
2. For each plan, check validity and budget.
3. Calculate base price plus excess usage charges.
4. Keep plans satisfying constraints.
5. Select and display the cheapest eligible plan.

Flowchart (text representation)

Start → Read usage/budget → Evaluate each plan → Calculate cost/eligibility → Select lowest cost → Display → End

Python source code

```
usage_gb = 18
required_days = 28
budget = 500

plans = [
    {"name": "Starter", "price": 249, "data": 12, "days": 28, "extra_rate": 20},
    {"name": "Value", "price": 399, "data": 25, "days": 28, "extra_rate": 15},
    {"name": "Flex", "price": 449, "data": 30, "days": 56, "extra_rate": 10},
]

eligible = []
for p in plans:
    excess = max(0, usage_gb - p["data"])
    total = p["price"] + excess * p["extra_rate"]
    valid = p["days"] >= required_days and total <= budget
    print(f'{p["name"]}: total Rs.{total:.2f} | Eligible: {valid}')
    if valid:
        eligible.append((total, p["name"]))

if eligible:
    cost, name = min(eligible)
    print(f"Best eligible plan: {name} at Rs.{cost:.2f}")
else:
    print("No plan meets the requirements.")
```

Sample input

Usage 18 GB; validity required 28 days; budget Rs.500.

Sample output

```
Starter: total Rs.369.00 | Eligible: True\nValue: total Rs.399.00 | Eligible: True\nFlex: total Rs.449.00 | Eligible: True\nBe
```

Result

The least expensive plan satisfying validity and budget conditions is selected. Plan prices and charges are sample data.

6. Railway Fare and Reservation Calculator

Problem statement

Calculate ticket fares for passengers based on distance, class multiplier, concession eligibility, reservation fee, and cancellation penalty.

Algorithm

1. Read distance and passenger details.

2. Calculate base fare using distance rate and class factor.

3. Apply the illustrative age concession and add reservation fee.

4. For cancellations, calculate the specified penalty.

5. Print passenger-wise fare details.

Flowchart (text representation)

Start → Read journey/passenger data → Compute base fare → Apply concession and fee → Optional cancellation penalty → Print total → End

Python source code

```
distance_km = 240
reservation_fee = 40
class_factor = 1.5       # illustrative class multiplier
rate_per_km = 1.2
cancellation_rate = 0.20
passengers = [
    {"name": "Anu", "age": 17, "cancel": False},
    {"name": "Dev", "age": 35, "cancel": True},
]

for p in passengers:
    base = distance_km * rate_per_km * class_factor
    concession = 0.50 if p["age"] < 12 else 0.00  # sample rule only
    fare = base * (1 - concession) + reservation_fee
    if p["cancel"]:
        penalty = fare * cancellation_rate
        refund = fare - penalty
        print(f'{p["name"]}: Fare Rs.{fare:.2f}, penalty Rs.{penalty:.2f}, '
              f'refund Rs.{refund:.2f}')
    else:
        print(f'{p["name"]}: Fare payable Rs.{fare:.2f}')
```

Sample input

Distance 240 km; class factor 1.5; rate Rs.1.20/km; reservation Rs.40; cancellation penalty 20%.

Sample output

```
Anu: Fare payable Rs.472.00\nDev: Fare Rs.472.00, penalty Rs.94.40, refund Rs.377.60
```

Result

The example demonstrates fare calculation and cancellation deductions. Age concessions and fare rates are illustrative and must be replaced by the applicable railway rules.

7. Retail Billing with Multiple Discount Rules

Problem statement

Create an itemized invoice for products, quantities, category-specific tax, threshold discount, and a maximum concession limit.

Algorithm

1. Read or define cart items and validate quantity/prices.
2. Calculate line totals and category taxes.
3. Calculate subtotal and threshold discount.
4. Cap the discount at the policy maximum.
5. Print itemized invoice and final amount.

Flowchart (text representation)

Start → Read cart → Compute line totals/taxes → Calculate capped discount → Find grand total → Print invoice → End

Python source code

```
cart = [
    {"name": "Notebook", "qty": 3, "price": 60, "tax": 0.05},
    {"name": "Headphones", "qty": 1, "price": 900, "tax": 0.18},
]
subtotal = 0
tax_total = 0
print("ITEMIZED INVOICE")
for item in cart:
    if item["qty"] < 0 or item["price"] < 0:
        raise ValueError("Quantity and price cannot be negative")
    line = item["qty"] * item["price"]
    tax = line * item["tax"]
    subtotal += line
    tax_total += tax
    print(f'{item["name"]}: {item["qty"]} x Rs.{item["price"]} '
          f'= Rs.{line:.2f}; tax Rs.{tax:.2f}')

discount_rate = 0.10 if subtotal >= 1000 else 0
discount = min(subtotal * discount_rate, 150)  # maximum concession
total = subtotal + tax_total - discount
print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Tax: Rs.{tax_total:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"GRAND TOTAL: Rs.{total:.2f}")
```

Sample input

3 notebooks at Rs.60 (5% tax) and 1 headphone at Rs.900 (18% tax).

Sample output

```
ITEMIZED INVOICE\nNotebook: 3 x Rs.60 = Rs.180.00; tax Rs.9.00\nHeadphones: 1 x Rs.900 = Rs.900.00; tax Rs.162.00\nSubtotal: R
```

Result

The invoice includes category tax and a threshold discount capped at Rs.150. Tax rates are examples only.

8. Manufacturing Cost and Profitability Analysis

Problem statement

Calculate material, wastage, labour, and machine costs; determine unit cost, selling revenue, profit, and whether a target profit margin is achieved.

Algorithm

1. Read production quantity and cost components.

2. Add material wastage allowance.

3. Calculate total manufacturing cost.

4. Calculate sales revenue and profit margin.

5. Compare margin with the target and report result.

Flowchart (text representation)

Start → Read production/cost values → Add wastage → Compute total cost/revenue → Find margin → Check target → Display → End

Python source code

```
quantity = 500
material_cost_per_unit = 80
wastage_rate = 0.05
labour_total = 12000
machine_total = 8000
selling_price_per_unit = 125
target_margin_percent = 15

if quantity <= 0:
    raise ValueError("Quantity must be positive")
material = quantity * material_cost_per_unit
wastage = material * wastage_rate
total_cost = material + wastage + labour_total + machine_total
revenue = quantity * selling_price_per_unit
profit = revenue - total_cost
margin = profit / revenue * 100 if revenue else 0

print(f"Material: Rs.{material:.2f}")
print(f"Wastage: Rs.{wastage:.2f}")
print(f"Total cost: Rs.{total_cost:.2f}")
print(f"Revenue: Rs.{revenue:.2f}")
print(f"Profit: Rs.{profit:.2f}")
print(f"Profit margin: {margin:.2f}%")
print("Target met" if margin >= target_margin_percent else "Target not met")
```

Sample input

500 units; material Rs.80/unit; wastage 5%; labour Rs.12000; machine Rs.8000; selling price Rs.125/unit; target margin 15%.

Sample output

```
Material: Rs.40000.00\nWastage: Rs.2000.00\nTotal cost: Rs.62000.00\nRevenue: Rs.62500.00\nProfit: Rs.500.00\nProfit margin: 0
```

Result

Production earns a small profit but does not reach the specified 15% margin target.

9. Multi-Currency Travel Budget Analysis

Problem statement

Convert travel expenses from multiple currencies using supplied rates, add conversion fees, compare the total with a budget, and identify the largest expense category.

Algorithm

1. Read expenses, currencies, and exchange rates.

2. Convert each expense to the home currency.

3. Add the conversion fee.

4. Accumulate category totals and overall spend.

5. Compare against budget and identify the largest category.

Flowchart (text representation)

Start → Read expenses/rates → Convert each amount → Add fees → Sum by category → Compare budget/find largest → Display → End

Python source code

```
# Exchange rates: units of INR per 1 foreign currency unit.
rates = {"USD": 83.0, "EUR": 90.0}
expenses = [
    {"category": "Hotel", "amount": 200, "currency": "USD"},
    {"category": "Food", "amount": 80, "currency": "EUR"},
    {"category": "Transport", "amount": 50, "currency": "USD"},
]
fee_rate = 0.02
budget_inr = 25000
category_totals = {}
total = 0

for e in expenses:
    if e["currency"] not in rates:
        raise ValueError("Exchange rate missing")
    converted = e["amount"] * rates[e["currency"]]
    fee = converted * fee_rate
    final = converted + fee
    category_totals[e["category"]] = final
    total += final
    print(f'{e["category"]}: Rs.{final:.2f} including fee')

largest = max(category_totals, key=category_totals.get)
print(f"Total spending: Rs.{total:.2f}")
print(f"Budget: Rs.{budget_inr:.2f}")
print("Within budget" if total <= budget_inr else "Over budget")
print(f"Largest category: {largest} (Rs.{category_totals[largest]:.2f})")
```

Sample input

Hotel USD 200; food EUR 80; transport USD 50; rates USD=Rs.83 and EUR=Rs.90; conversion fee 2%; budget Rs.25000.

Sample output

```
Hotel: Rs.16932.00 including fee\nFood: Rs.7344.00 including fee\nTransport: Rs.4233.00 including fee\nTotal spending: Rs.2850
```

Result

Total spending exceeds the example budget, and Hotel is the largest expense category. Exchange rates are sample values; update them before use.

10. Campus Event Financial Planning

Problem statement

Calculate fixed and per-participant costs for venue, equipment, catering, transport, and certificates; apply sponsorship and determine contribution and break-even registration count.

Algorithm

1. Read fixed expenses and variable cost per attendee.
2. Calculate total cost for expected registrations.
3. Subtract sponsorship to find funding gap.
4. Calculate required contribution per participant.
5. Compute minimum registrations needed to break even at the planned fee.

Flowchart (text representation)

Start → Read event expenses/sponsor/fee → Calculate cost → Determine funding gap → Calculate break-even count → Print plan → End

Python source code

```
venue = 12000
equipment = 5000
transport = 6000
catering_per_person = 180
certificate_per_person = 20
sponsorship = 10000
registration_fee = 300
expected_registrations = 100

fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * expected_registrations
funding_gap = max(0, total_cost - sponsorship)
participant_contribution = funding_gap / expected_registrations

# Minimum registrations to cover fixed costs after sponsorship
# plus variable expenses using registration fees.
net_fixed = max(0, fixed_cost - sponsorship)
surplus_per_person = registration_fee - variable_per_person
if surplus_per_person <= 0:
    break_even = None
else:
    break_even = (net_fixed + surplus_per_person - 1) // surplus_per_person

print(f"Fixed costs: Rs.{fixed_cost:.2f}")
print(f"Variable cost/person: Rs.{variable_per_person:.2f}")
print(f"Total cost: Rs.{total_cost:.2f}")
print(f"Sponsorship: Rs.{sponsorship:.2f}")
print(f"Funding gap: Rs.{funding_gap:.2f}")
print(f"Required contribution/person: Rs.{participant_contribution:.2f}")
print("Break-even registrations:", break_even if break_even is not None
      else "Not possible at this fee")
```

Sample input

Venue Rs.12000; equipment Rs.5000; transport Rs.6000; catering Rs.180/person; certificate Rs.20/person; sponsorship Rs.10000; fee Rs.300; planned attendees 100.

Sample output

```
Fixed costs: Rs.23000.00\nVariable cost/person: Rs.200.00\nTotal cost: Rs.43000.00\nSponsorship: Rs.10000.00\nFunding gap: Rs.
```

Result

At Rs.300 per registration and Rs.200 variable cost per attendee, 115 registrations are required to cover the remaining fixed cost after sponsorship.